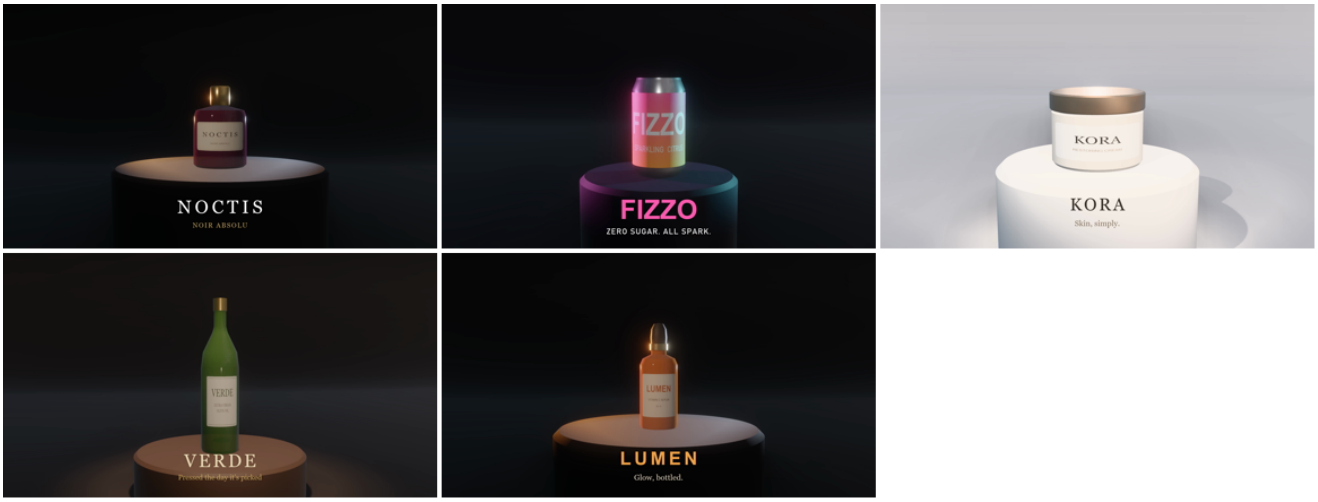
AI Product Ad Studio

Meta Global AI Developer Hackathon · project overview: architecture, modules, dependencies, status

A multi-agent pipeline that turns a product photo and a short brief ("luxury perfume ad, dark and moody, 15 seconds") into a finished, studio-quality 3D marketing video. A brand-analyst agent reads the product with SAM 3.1 and Muse Spark; a director agent plans a storyboard and drives a physically based renderer through tool calls; a critic agent inspects the rendered frames, scores them against a rubric and sends specific fixes back until every shot passes. The goal: professional product marketing for people who could never afford a studio.

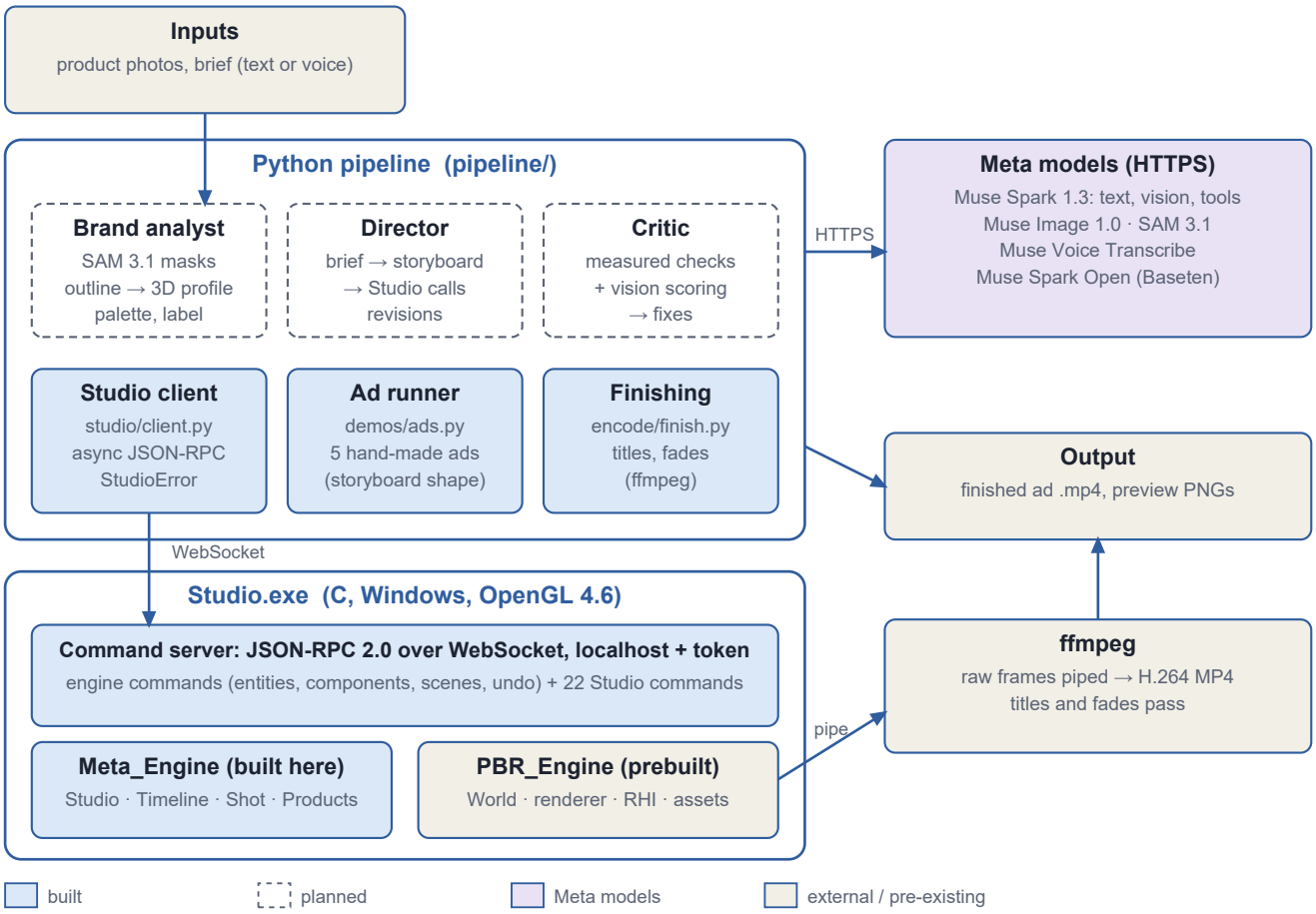
Status (2026-10-28)	Renderer side complete through roadmap Phase 1.6, a day ahead of schedule. Agents next.
Track · problem statement	Indie Developer · Build for Your World
Repository	github.com/Mar-Dev-One (Meta-hackathon); the engine: AI-Powered-PBR-Renderer
Code written so far	≈ 4,900 lines: 4,100 C (Meta_Engine + Studio + tests), 1,000 Python (pipeline)
Deadline	Wed Nov 4, 9:00 PM ET (target: submitted by 6 PM ET)



Final frames of the five hand-made test ads (fictional brands), rendered by the Studio at 1080p: Noctis perfume, Fizzo drink can, Kora skincare jar, Verde olive oil, Lumen serum.

1. Architecture

Three parts talk over two interfaces. The Python pipeline holds the agents; the Studio is a C program that renders; Meta's models are called over HTTPS. The pipeline drives the Studio only through its command server, so the two run, restart and develop independently.



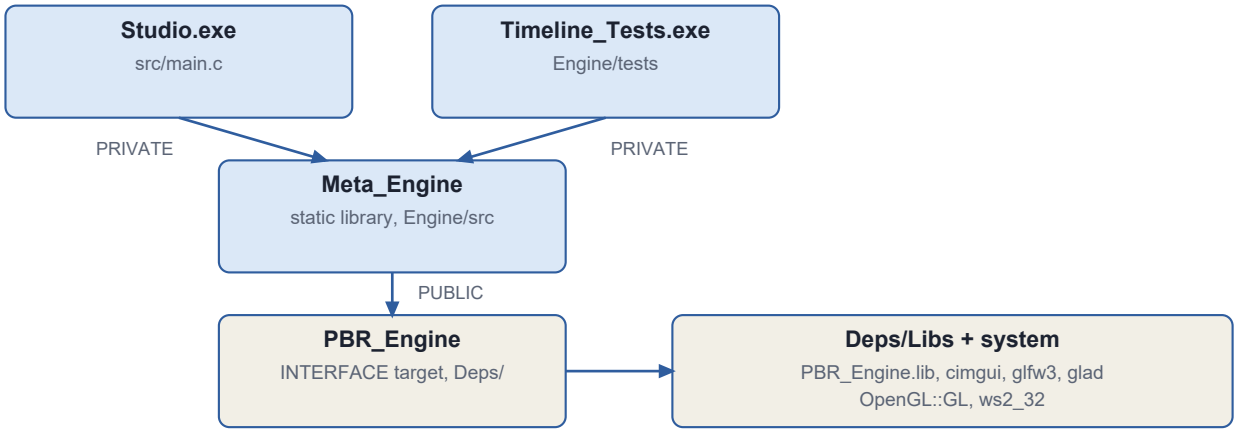
Principles

- **Deterministic frames.** A frame is a function of the scene, the timeline and its time. Rendering a frame alone gives the same pixels as rendering it in a sequence, so the critic can re-check exactly one frame.
- **Agents never compute camera maths.** They pick named moves, lighting presets and product templates and fill in a few numbers; the Studio turns those into geometry and keys.
- **Everything is validated.** Every command checks its arguments before changing anything; error messages say which argument is wrong and what it accepts, written for a model to correct itself.
- **Measure before judging.** The critic will combine exact checks from the renderer (framing, cut edges, light balance) with vision scoring, so its verdicts are grounded.
- **The engine is unchanged.** Everything new is in this repository, built on the engine's public headers.

2. Repository layout

```
Meta-hackathon/
  src/main.c                Studio.exe: window or --hidden, command server, frame loop
  Engine/                   Meta_Engine (static library, written for this project)
    src/Common/             JSON helpers and argument readers shared by every module
    src/Studio/             studio state, environment/render/camera commands,
                            camera moves, lighting presets, look-at, scene save/load
    src/Timeline/           keyframes on any component field, easing, commands
    src/Shot/               offscreen frames, TAA restart, PNG, ffmpeg video, framing checks
    src/Products/           lathe/extrusion geometry, glTF writer, product and backdrop commands
    tests/timeline_tests.c  C tests (ctest)
  pipeline/                 Python
    studio/client.py        async client for the Studio's command server
    encode/finish.py        ffmpeg finishing: titles, fades
    demos/ads.py            spec-driven ad runner, five hand-made ads
    meta/smoke/             one test call to each Meta model
  Deps/                     the engine, prebuilt: headers (Includes/PBR) and MSVC libraries (Libs)
  Assets/                   engine shaders, demo model, HDRI (copied next to the exe at build)
  docs/ROADMAP.md           the plan, step by step
```

2.1 Build dependencies (CMake)



PBR_Engine is an INTERFACE target: it compiles nothing and packages the prebuilt libraries, system libraries and the header folder. **Meta_Engine** links it PUBLIC, so anything linking Meta_Engine gets the engine too. The build copies shaders, models and HDRIs into `build/Release/assets` and `vcruntime140.dll` next to the exe, so the build folder runs on any Windows 10/11 PC with an OpenGL 4.x driver.

2.2 Inside Meta_Engine

Module	Depends on	Role
Common/Json	cJSON	set_error, rounding, float arrays, optional-argument readers
Timeline	World, Reflect, cJSON	Tracks over any number / vec3 / colour field; easing; JSON. No dependency on the Studio: testable without a GPU
Studio	Timeline, Shot, IBL, SceneRenderer	Owns the timeline and shot renderer; studio_evaluate(t): timeline → look-at aim → world update; scene sidecar file
Shot	Studio, SceneRenderer, Image	Own renderer at any size; frames to PNG or ffmpeg; shot.bounds
Products	Studio, Encoding, FileIO	Lathe and extrusion meshes written as glTF and loaded by the engine's own loader

3. Dependencies

3.1 C side

Dependency	Version / source	Used for
PBR_Engine	own engine, prebuilt in Deps/ (pre-hackathon)	World (entities, components, reflection), PBR renderer (IBL, clustered lights, shadows, SSAO, TAA, bloom, AgX), asset cache, glTF/OBJ loaders, command layer with undo, WebSocket command server
GLFW, glad	linked via Deps/Libs	window, OpenGL 4.6 context and loader
cimgui	linked via Deps/Libs	Dear ImGui (engine panels; not used by the Studio yet)
cglm	header-only, Deps/Includes	vector and matrix maths
cJSON	Deps/Includes (compiled into the engine)	every command's JSON
stb_image / stb_image_write	inside the engine	image decode, PNG encode
OpenGL, ws2_32	Windows	graphics; sockets for the command server
Toolchain	MSVC (VS 2022), CMake ≥ 3.26	/MD everywhere: the libraries are Release builds

3.2 Python side (pipeline/requirements.txt)

Package	Used for
openai ≥ 1.60	Meta Model API (OpenAI-compatible, api.meta.ai/v1) and Baseten (inference.baseten.co/v1)
websockets ≥ 12	the Studio client (ping timeout off: renders block the Studio)
meta-sam-parser ≥ 0.1	decoding SAM 3.1's text output: boxes and base85 mask rasters
pillow ≥ 10	label artwork, contact sheets, mask overlays
python-dotenv, requests	API keys from .env (git-ignored); the voice endpoint's multipart upload

3.3 Tools and services

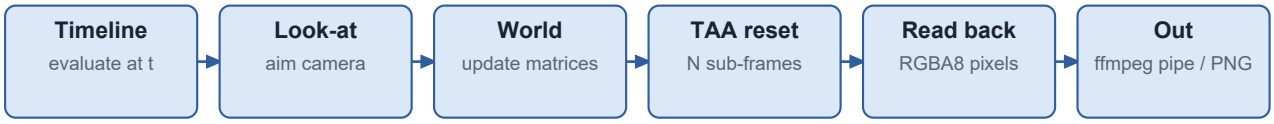
Tool / service	Used for
ffmpeg (on PATH)	Studio pipes raw frames into H.264; the finishing pass adds titles and fades
Meta Model API	Muse Spark 1.3, Muse Image 1.0, SAM 3.1, Muse Voice Transcribe 1.0 ($150 credits)
Baseten	Muse Spark Open (meta-models/muse-spark-open-1026): the open-weight critic (key pending)

4. The Studio

Studio.exe starts on an empty stage (or `--demo-scene`), optionally hidden, at any size. It runs the engine's command server and installs the Studio's commands next to the engine's: entities, components, scenes, batches and undo come from the engine; the 22 below are new.

Command	What it does
studio.reset	Empty stage: entities, tracks, environment, render settings, camera
environment.get / .set	HDRI, image-based light, exposure, bloom, tone mapping (AgX, ACES, Neutral), SSAO
render.get / .set	Antialiasing (TAA by default), shadows, background colour
camera.get / .set	Creates or moves a camera, aims it once, makes it active
camera.move	A shot's move around a subject: static, orbit, dolly, crane, truck, reveal, zoom. Framed by fill; written as keys over its own time range, so moves end to end are cuts
lighting.preset	three_point, rim_dark, high_key, top_soft, split, neon_dual, placed by the subject's size; intensity, turn and warmth adjustments; lights are light_* entities
product.create	A product spun from a profile: perfume, bottle, jar, can, dropper or custom parts; "#RRGGBB" colours; a label image wrapped round the body
backdrop.create	A cyclorama (floor curving into a wall) or a round pedestal, optionally textured
timeline.set_track / add_key / remove_track / clear / get	Keys on any number, vec3 or colour field of any component, with linear, in, out, in_out, step or bezier easing; values checked against each field's range
timeline.seek / play / pause	Show a time; real-time preview in the window
shot.preview	PNG frames at given times (the critic's input)
shot.render	A time range at an fps, straight into an MP4 through ffmpeg (or PNG frames)
shot.bounds	Where an entity sits in the frame: box, centre, fill, which edges cut it

4.1 How a frame is rendered

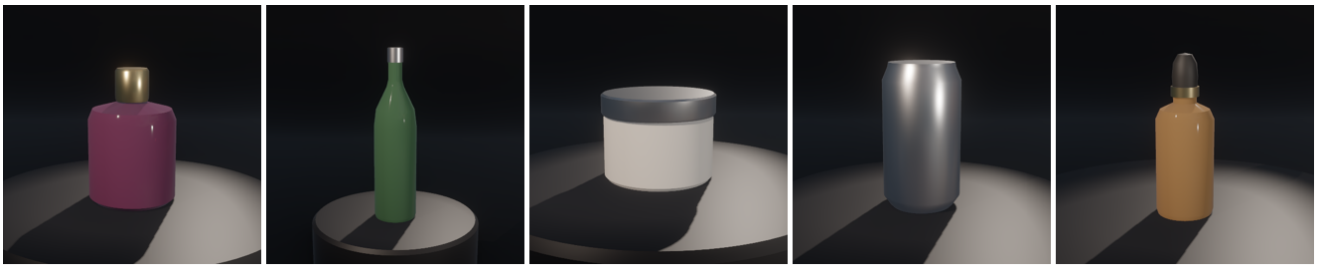


Each output frame evaluates the timeline at its own time, aims the camera at its look-at target, and renders N jittered sub-frames with TAA starting from a clean history, then reads the pixels back. On an Intel Iris Xe a 1080p frame with 16 samples takes about 0.4 s into a video (a 15-second ad: about 3 minutes); PNG output costs about 1.2 s per 1080p frame, mostly compression, so it's kept for previews.

4.2 Products as glTF

Generated products and backdrops are written as glTF files under the project's `products/` folder and loaded by the engine's own glTF loader. That keeps them working across scene saves and Studio restarts, gets sRGB textures, transparency and metal right for free, and lets them open in Blender. A file's name carries a hash of its request: the same request reuses its file.

4.3 Products, labels and lighting presets



The five product templates, lit by three_point.



One perfume with a wrapped label under rim_dark, high_key, neon_dual and split; the last frame was reloaded from a saved scene in a fresh Studio.



One shot of each camera move, placed end to end (orbit, dolly, crane, reveal, truck, zoom, static).

5. Pipeline and Meta models

5.1 Built so far

- **Studio client** (`pipeline/studio`): starts Studio.exe hidden, waits for this run's server.json (matched by process id), runs commands, raises StudioError(code, message) on failure.
- **Finishing** (`pipeline/encode`): titles with fades, letter spacing and colour; fades from and to black. Moves into the Studio in Phase 1.7 so the critic can see titles.
- **Ad runner** (`pipeline/demos/ads.py`): five ads as specs (scene, product, label, lighting, shots, extra tracks, titles): the shape of what the director will write.

5.2 Smoke tests against the Meta Model API

Model	Result	Time
Muse Spark 1.3 (text)	Works	4.8 s
Muse Spark 1.3 (tools)	3 sensible calls over 2 turns (camera, then two lights in parallel), then a description of the shot	36 s
Muse Image 1.0	Clean fictional product photo; size sets the aspect (1024×1536 → 1280×1920)	9.7 s
Muse Spark 1.3 (vision)	Valid JSON: parts, metallic/transparent finishes, palette, typography, mood	7.3 s
SAM 3.1	Pixel-accurate masks for bottle, cap, label; large masks come at ≤1024 px and are scaled to their box	3-8 s
Muse Voice Transcribe	Works (a synthetic voice gave a rough transcript)	3.9 s
Muse Spark Open (Baseten)	Pending the Baseten key	–



Muse Image's product photo and SAM 3.1's masks (bottle, cap, label). The bottle's outline gives a lathe profile: 37 px at the base, 424 px across the body, 147 px at the neck.

6. Verification

Check	What it covers	Result
Timeline_Tests (ctest)	Easing curves, sampling, editing, range checks, writing into a World, JSON	pass
Studio commands, end to end	Valid calls, bad values, unknown arguments, undo	pass
Timeline, end to end	Eased values at given times, errors, save/load with the scene, playback	25 / 25
Shots, end to end	Identical PNGs for the same time; frame 15 of a sequence = t = 0.5 alone; bounds; timeline restored	16 / 16
Camera moves, end to end	Every move centred and uncut; cuts between moves; save/load	30 / 30
Products, end to end	Templates, labels, errors, reload in a fresh Studio from glTF files	11 / 11
Rendered output	Contact sheets and videos inspected by eye at each step	–

The end-to-end scripts drive a real Studio over WebSocket. They live outside the repository so far; moving them into `tests/` is a small, worthwhile step.

Bugs found and fixed along the way

- Out-of-range timeline keys were silently clamped while the engine's component.set rejects them: now rejected the same way.
- camera.set allowed a field of view and near plane beyond the Camera component's own ranges.
- PNG compression was the render bottleneck (not the GPU): video now streams to ffmpeg, three times faster.
- A long render blocks the Studio's WebSocket pings: clients turn off the ping timeout.
- SAM's large masks arrive downscaled; the label hash read a PNG only up to its first zero byte; flipped extrusion normals.

7. Built during the hackathon vs before

Before the hackathon (from 2026-07-21)	During the hackathon (from 2026-10-27)
The PBR engine (AI-Powered-PBR-Renderer): renderer, World, assets, command layer and server. Used unchanged, as a prebuilt library.	Everything in this repository: Meta_Engine (Studio, Timeline, Shot, Products, Common), Studio.exe, the Python pipeline, tests, demos, docs. First commit 2026-10-27 11:15.

Every brand in the demos is fictional. Test titles use Windows system fonts; shipped demos will use open-licensed fonts. The organizers have been asked to confirm that a solo participant may use their own pre-existing engine as a dependency.

8. Roadmap

Remaining work is split into steps of about 1-2 hours; each ends with something checked, and work stops for a review after each one.

Phase	Steps	When
2 Pipeline foundation	2a Meta clients (cache, retries, costs) · 2b schemas · 2c run folders	Oct 28
3 Brand analyst	3a segmentation · 3b outline → 3D profile · 3c label and palette · 3d materials, mood · 3e brand profile	Oct 28-29
1.7 Text in the Studio	fonts and atlas · titles in the frame · animated text · ads moved over	Oct 30
4 Director	4a grammar and styles · 4b brief → storyboard · 4c storyboard → Studio · 4d tools and revisions · 4e first agent-made video · 4f voice brief	Oct 30-Nov 1
5 Assets	5a backdrop textures · 5b labels, end cards · 5c quality gate (Muse Image)	Nov 1
6 Critic	6a measured checks · 6b vision scoring · 6c fix loop · 6d continuity · 6e Muse Spark Open	Nov 1-2
7 Final cut, app	7a final render, music, 9:16 · 7b backend · 7c web page · 7d replay	Nov 2-3
8 Freeze	8a three demo ads · 8b evaluation numbers · 8c README, release	Nov 3
9 Submission	9a 3-minute video · 9b submit by 6 PM ET, X post	Nov 4

Risks and fallbacks

Risk	Fallback
Rendering is slow on integrated graphics	Low-resolution previews for the critic; final renders overnight
Glass has no refraction	Dark, frosted or opaque products; rim light sells the shape
Silhouette-to-profile fails on a photo	Category template, label still from the photo
Tool calling unreliable	Storyboard JSON compiled to commands in code (4c)
API outage or credits run out	Disk cache, replayable run folders, the hand-made ads